

Alaa Abdelqader

This report explores the establishment of a basic development environment using Gitflow. The main objective is to apply the Gitflow workflow... incorporating feature branches, a development branch, and a main branch. Additionally, the report details the execution of release and hotfix processes throughout the project lifecycle. The project is initiated by employing the 'git init' command, creating a Git repository to oversee the codebase.

Gitflow Implementation: The Gitflow workflow is structured around three main types of branches: feature branches, release branches, and hotfix branches.

- **Feature branches**

Feature branches are created to isolate the development of new features. Developers work independently on these branches, and once the feature is ready, it is merged back into the development branch.

```
git checkout develop    —> Switch to the development branch
git checkout -b feature_branch —> Create and switch to a new feature branch
git merge feature_branch —> Merge the feature branch into the development branch
git branch -d feature_branch —> Delete the feature branch
git push origin develop —> Push changes to the remote repository
```

- **Release Branches**

Release branches are created from the development branch and serve as an isolated environment for finalizing features, conducting testing, and addressing last-minute issues before the software release

Creating a Release branch

git checkout develop → Switch to the development branch
git checkout -b release/0.1.0 → Switched to the new branch

Finishing a feature branch

git checkout main → Switched to branch

git merge release/0.1.0

git branch -d release/0.1.0 → if i want to delete the branch

- **Hotfix Branches**

Imagine hotfix branches as your software's emergency repair kit. They're a lot like release branches but come into play when something crucial in your software needs an urgent fix. If there's a significant issue in the version of the software currently in use by customers and it demands an immediate solution, that's when you create a hotfix branch. It's a quick patch-up, like fixing a critical bug or resolving a major problem, ensuring that your users get a smoother experience without delay.

```
git checkout main → Switch to the main branch
git checkout -b hotfix1 → Create and switch to a new hotfix branch
git merge hotfix1 → Merge the hotfix branch into both main and develop
git checkout develop → Switch back to the development branch
git merge hotfix1 → Merge the hotfix branch into the development branch
git branch -d hotfix1 → Delete the hotfix branch
```

Challenges: I encountered difficulties like resolving conflicts during merging and determining the right timing for creating specific branches. To overcome these challenges, I continually worked through issues, dedicating time to grasp the intricacies of Gitflow. This process was particularly important since I'm working independently without a team.

Learning Experience :

Using what I learned in the "New Batic Project" and trying out GitFlow in real situations has been a big learning experience. It's not just about understanding how to control versions and work well with a team; it's about putting those ideas into action. Figuring out the structured development, isolating features, and quickly fixing problems and applying quick fixes are more than just lessons now—they're habits. This experience will really change how I do version control and teamwork in my future projects. It shows that I've gotten better at handling changes in the code and working well with a team.